

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。 `/daily` と打つだけでデイリーノートが生成される。 `/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか

```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で **スラッシュコマンド** として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 **毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった**。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```

ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイル进行分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code` の最新機能 と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

`commands/` が「何をするか」なら、`rules/` は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: `tech-stack.md` — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか?」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: `naming.md` — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: `timezone.md` — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: `security.md` — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

`rules/` にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: `cascade.md` — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```

- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

```
# vault-master SKILL.md
```

```
## 概要
```

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

```
## ワークフロー
```

```
### Step 1: 現状分析
```

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

```
### Step 2: Inbox 整理
```

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

```
### Step 3: アーカイブ
```

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`

```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync ― タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要なら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。`/daily` と打つだけでデイリーノートが生成される。`/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか


```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で **スラッシュコマンド** として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 **毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった**。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```

ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイル进行分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code` の最新機能 と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

`commands/` が「何をするか」なら、`rules/` は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: tech-stack.md — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか?」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: naming.md — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: timezone.md — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: security.md — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

rules/ にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: cascade.md — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```

- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

vault-master SKILL.md

概要

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

ワークフロー

Step 1: 現状分析

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

Step 2: Inbox 整理

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

Step 3: アーカイブ

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`

```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync — タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要なら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。`/daily` と打つだけでデイリーノートが生成される。`/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか

```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で **スラッシュコマンド** として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 **毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった**。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```


ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイルを分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code` の最新機能 と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

commands/ が「何をするか」なら、rules/ は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: `tech-stack.md` — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか?」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: `naming.md` — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: `timezone.md` — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: `security.md` — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

`rules/` にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: `cascade.md` — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```

- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

```
# vault-master SKILL.md
```

```
## 概要
```

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

```
## ワークフロー
```

```
### Step 1: 現状分析
```

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

```
### Step 2: Inbox 整理
```

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

```
### Step 3: アーカイブ
```

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`

```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync ― タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要なら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。`/daily` と打つだけでデイリーノートが生成される。`/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド (35個)
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準 (18個)
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー (10個)
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか

```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で **スラッシュコマンド** として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 **毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった**。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```

ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイル进行分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code` の最新機能 と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

`commands/` が「何をするか」なら、`rules/` は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: `tech-stack.md` — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか?」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: `naming.md` — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: `timezone.md` — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: `security.md` — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

`rules/` にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: `cascade.md` — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```

- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

```
# vault-master SKILL.md
```

```
## 概要
```

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

```
## ワークフロー
```

```
### Step 1: 現状分析
```

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

```
### Step 2: Inbox 整理
```

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

```
### Step 3: アーカイブ
```

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`

```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync ― タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要なら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。 `/daily` と打つだけでデイリーノートが生成される。 `/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/          # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか

```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で **スラッシュコマンド** として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 **毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった**。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```

ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイル进行分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code の最新機能` と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

`commands/` が「何をするか」なら、`rules/` は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: `tech-stack.md` — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか?」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: `naming.md` — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: `timezone.md` — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: `security.md` — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

`rules/` にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: `cascade.md` — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```


- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

```
# vault-master SKILL.md
```

```
## 概要
```

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

```
## ワークフロー
```

```
### Step 1: 現状分析
```

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

```
### Step 2: Inbox 整理
```

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

```
### Step 3: アーカイブ
```

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`

```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync ― タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要なら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。`/daily` と打つだけでデイリーノートが生成される。`/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか

```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で スラッシュコマンド として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```

ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイル进行分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code` の最新機能 と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

`commands/` が「何をするか」なら、`rules/` は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: tech-stack.md — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか?」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: naming.md — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: timezone.md — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: security.md — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

rules/ にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: cascade.md — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```

- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

vault-master SKILL.md

概要

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

ワークフロー

Step 1: 現状分析

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

Step 2: Inbox 整理

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

Step 3: アーカイブ

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`


```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync ― タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要なら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。 `/daily` と打つだけでデイリーノートが生成される。 `/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか

```
# My Obsidian Vault

## このVaultについて
- 個人のナレッジマネジメント用Obsidian Vault
- Daily Notes、タスク管理、プロジェクトメモを集約
- 外部ツール（Azure DevOps, OneNote）とMCP経由で連携

## ディレクトリ構成
- `00_Inbox/` - 未整理のメモ・アイデア
- `01_Projects/` - プロジェクト単位のノート
- `02_Daily/` - Daily Notes（YYYY/YYYY-MM/YYYY-MM-DD/）
- `03_Archive/` - 完了・非アクティブなノート
- `04_Templates/` - テンプレートファイル

## 命名規則
- ファイル名は日本語OK
- Daily Notesのパス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`

## よく使うコマンド
- `/daily` - 今日のデイリーノート作成
- `/weekly-review` - 週次レビュー
- `/inbox-review` - Inbox整理

## 注意事項
- タイムゾーンは必ず Asia/Tokyo で確認すること
- ファイル削除は `trash` を使う（`rm` 禁止）
```

何を書かないべきか

ここが結構大事で。CLAUDE.md に なんでもかんでも書くと、コンテキストウィンドウを圧迫して性能が落ちる。公式ドキュメントでも「300行以内」が推奨されている。だから、以下は CLAUDE.md に書かずに別ファイルに分ける。

書かないもの	分け先	理由
詳細なルール	rules/	トピック別に分けたほうが管理しやすい
操作手順	commands/	コマンドとして呼び出せたほうが便利
複合ワークフロー	skills/	手順が長すぎてCLAUDE.mdが肥大化する
機密情報	書かない	APIキー等は絶対に含めない

CLAUDE.md は「概要と目次」 だと思うといい。詳細は別ファイルに任せて、CLAUDE.md 自体はスリムに保つ。

commands/ — 日常操作をスラッシュコマンド化する

.claude/commands/ に .md ファイルを置くと、Claude Code のセッション内で **スラッシュコマンド** として呼び出せる。

例えば .claude/commands/daily.md を作ると、 /daily と打つだけでそのファイルの内容が実行される。

これがめっちゃ便利で。僕の場合、 **毎朝5分かかっていたデイリーノート作成が、10秒で終わるようになった**。

実例1: /daily — デイリーノート自動生成

```
# デイリーノート生成

以下の手順でデイリーノートを作成してください。

1. 現在の日付を `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` で確認
2. `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/` ディレクトリを作成
3. `04_Templates/daily-template.md` を読み込み
4. テンプレートの日付・曜日を置換
5. `02_Daily/tasklist/doing/` のファイルを読んで、進行中タスクを反映
6. 曜日に応じたスケジュールを追加（平日はルーティン、土日はフリー）
7. 作成したノートのサマリーを報告
```

ポイントは **自然言語で書けること**。コードじゃなくて、「こうしてほしい」という指示を日本語で書くだけでいい。

実例2: /weekly-review — 週次レビュー

```
# 週次レビュー

今週の振り返りを行います。

1. 今週の Daily Notes (02_Daily/YYYY/YYYY-MM/ 配下の直近7日分) を読む
2. 完了したタスク、未完了のタスク、新たに発生した課題を整理
3. KPT (Keep / Problem / Try) 形式でレビューをまとめる
4. 来週に持ち越すタスクを `02_Daily/tasklist/todo/` に反映
5. レビュー結果を `02_Daily/YYYY/YYYY-MM/weekly-review-YYYY-MM-DD.md` に保存
```

これを毎週金曜に実行する。30分かかっていただ振り返りが、**Claude と対話しながら5分** で終わる。しかも、自分一人では気づかなかった「今週のパターン」を指摘してくれたりする。

実例3: /inbox-review — Inbox 分析・整理

```
# Inbox レビュー

`00_Inbox/` の全ファイル进行分析し、整理案を提示してください。

1. `00_Inbox/` の全ファイルを読む
2. 各ファイルの内容を分析し、以下に分類:
  - `01_Projects/` に移動すべきもの (プロジェクト関連)
  - `03_Archive/` に移動すべきもの (完了・不要)
  - 統合すべきもの (重複・関連ノート)
  - そのまま残すもの (まだ未整理でOK)
3. 分類結果を一覧で表示し、ユーザーの承認を待つ
4. 承認後、ファイルを移動 (`trash` で削除、`mv` で移動)
```

実例4: /mtg — 議事録テンプレ作成

```
# MTG議事録作成

$ARGUMENTS の議題名で議事録テンプレートを作成してください。

1. `04_Templates/mtg-template.md` を読み込む
2. 日時: 現在時刻 (JST)
3. 議題: $ARGUMENTS
4. 参加者: (空欄)
5. `01_Projects/meetings/` に保存
```

\$ARGUMENTS を使うと、 `/mtg プロジェクトAの進捗確認` のようにコマンドに引数を渡せる。これで議事録のファイル名と議題が自動で設定される。

実例5: /research — テーマ調査

```
# リサーチ

$ARGUMENTS のテーマについて網羅的に調査してください。

1. Web検索で最新情報を収集 (3-5ソース)
2. 各ソースの要点をまとめる
3. テーマに対する現状・課題・今後の展望を整理
4. `00_Inbox/research-{テーマ}-{日付}.md` に保存
5. 関連する既存ノートがあればリンクを提案
```

`/research Claude Code の最新機能` と打つだけで、Web検索して要点をまとめて、Vault内の関連ノートまで提案してくれる。これは本当に助かる。

rules/ — AIの「判断基準」を明文化する

`commands/` が「何をするか」なら、`rules/` は「どう判断するか」。

CLAUDE.md に全部書くとすぐに300行を超えてしまうので、トピック別に `.claude/rules/` にファイルを分ける。Claude Code は **タスクに関連するルールファイルを自動で読み込む** ので、必要なときに必要なルールだけがロードされる。

実例1: `tech-stack.md` — 技術スタックの制約

```
# 技術スタック

## 必須
- TypeScript / Next.js (App Router)
- MongoDB Atlas (データベース)
- Tailwind CSS

## 禁止
- Supabase (使わない)
- Prisma (使わない、Mongoose を使う)
- pages/ ルーター (App Router のみ)

## 理由
MongoDB + Mongoose を全プロジェクトで統一しているため。
技術選定の議論は不要。指示がない限りこのスタックで構築すること。
```

これがあると、Claude が勝手に「Supabase使いませんか？」と提案してこなくなる。 **暴走を防ぐガードレール** として機能する。

実例2: `naming.md` — ファイル命名規則

```
# 命名規則

- ファイル名は日本語OK (Obsidian の制約に合わせる)
- タスクファイル: `{プロジェクト名}-{タスク名}-{日付}.md`
- Daily Notes パス: `02_Daily/YYYY/YYYY-MM/YYYY-MM-DD/`
- 日付フォーマット: YYYY-MM-DD (ハイフン区切り)
```

実例3: `timezone.md` — タイムゾーン確認必須

```
# タイムゾーンルール

- 日時は必ず Asia/Tokyo (JST) 基準
- 曜日を確認するときは `TZ=Asia/Tokyo date "+%Y-%m-%d %A"` を実行すること
- 推測で曜日を判断しない (必ずコマンドで確認)
- 土日は常駐業務なし
```

これ、地味に重要で。AIって曜日を間違えることが結構ある。「今日は月曜日ですね」って言われて実際は火曜だったりする。だから **「推測禁止、必ずコマンドで確認」** というルールを明文化している。

実例4: `security.md` — セキュリティルール

```
# セキュリティルール

## 絶対に出力しないもの
- APIキー、トークン、パスワード
- .env ファイルの内容
- 個人情報 (住所、電話番号、カード番号)

## ファイル操作
- 削除は `trash` コマンドを使う (`rm` 禁止)
- 外部送信 (メール、SNS投稿) は必ず確認を取る
- 機密ファイルのパスを外部に共有しない
```

`rules/` にセキュリティルールを入れておくのは強くおすすめする。AIに Vault を触らせる以上、 **「やってはいけないこと」を明確に定義しておく** のが安全運用の基本になる。

実例5: `cascade.md` — 親子タスクの連動

```
# タスク連動ルール

- 親タスクを「Done」にしたら、子タスクも全て「Done」にする
```

- 子タスクが全て「Done」になったら、親タスクも「Done」を提案する
- タスクのステータス変更時は、関連タスクへの影響を報告すること

パススコープについて

rules/ のファイルは **パススコープ** を設定できる。つまり、「特定のディレクトリで作業しているときだけ有効になるルール」を作れる。

例えば、02_Daily/ 配下で作業しているときだけ適用されるルールを書きたければ、ルールファイルの冒頭にパス指定を入れる。

```
---
globs: ["02_Daily/**"]
---

# Daily Notes ルール

- テンプレートの形式を崩さない
- 「今日やったこと」「明日やること」「メモ」の3セクションを必ず含める
```

これで、Daily Notes 以外のファイルを編集しているときにはこのルールがロードされない。 **必要なときに必要なルールだけ**。コンテキストウィンドウの節約にもなる。

skills/ — 複雑なワークフローを「1つのフォルダ」にまとめる

commands/ との違いが最初わからなくて、正直混乱した。

結論から言うと、こういう使い分けになる。

	commands/	skills/
粒度	単一の操作	複数ステップの複合ワークフロー
ファイル	1つの .md	フォルダ (SKILL.md + 補助ファイル)
呼び出し	/コマンド名	関連タスク時に自動ロード
例	/daily (ノート1つ作る)	vault-master (Vault全体を整理)

commands/ は「ボタン1つで実行」、skills/ は「レシピブック」だと思えばいい。

実例: vault-master — Vault 全体の整理自動化

```
.claude/skills/vault-master/
├── SKILL.md           # メインの手順書
└── classification.md # 分類ルールの詳細
```

```
# vault-master SKILL.md
```

```
## 概要
```

Obsidian Vault 全体の整理・最適化を行うスキル。
月に1回の実行を推奨。

```
## ワークフロー
```

```
### Step 1: 現状分析
```

- 各ディレクトリのファイル数をカウント
- 3ヶ月以上更新のないファイルをリストアップ
- リンク切れを検出

```
### Step 2: Inbox 整理
```

- `00\_Inbox/` の全ファイルを classification.md のルールで分類
- 分類結果をユーザーに提示し、承認を得る

```
### Step 3: アーカイブ
```

- 承認されたファイルを `03\_Archive/` に移動
- 移動先のパス: `03\_Archive/YYYY-MM/`

```
### Step 4: リンク修復
- 移動したファイルへのリンクを自動更新
- 更新したリンクの一覧を報告

### Step 5: レポート
- 整理結果のサマリーを `02_Daily/` 配下に保存
- Before / After のファイル数を報告
```

実例: sprint-task-sync ― タスク同期ワークフロー

```
.claude/skills/sprint-task-sync/
├─ SKILL.md
└─ field-mapping.md    # フィールドのマッピング定義
```

```
# sprint-task-sync SKILL.md

## 概要
Obsidian のタスクファイルを Azure DevOps と OneNote に同期するスキル。

## ワークフロー
1. `02_Daily/tasklist/` のファイルを読み取り
2. Azure DevOps に PBI / Task を作成・更新（MCP経由）
3. OneNote の該当セクションに整形して登録
4. Obsidian 側にADのリンクを追記
5. 同期結果をレポート
```

commands/ が「1つの操作を自動化」するのに対して、skills/ は「複数のツールを横断する複雑なワークフロー全体」をパッケージ化する。

実運用で学んだ5つの設計原則

2ヶ月ほど運用して、いくつか「こうしておけばよかった」という学びがあったので共有する。

原則1: CLAUDE.md は「薄く」、rules/ を「厚く」

最初、CLAUDE.md に全部書いていた。ルールも手順も注意事項も。結果、500行を超えて、Claude の応答が明らかに遅くなった。

今は **CLAUDE.md は80行程度** に抑えて、詳細は全て rules/ に分けている。Claude Code は関連するルールファイルだけを動的にロードしてくれるので、これが一番効率がいい。

原則2: コマンドは「動詞 + 名詞」で命名する

最初 review.md check.md みたいな曖昧な名前をつけていた。でも20個を超えたあたりで、何がなんだかわからなくなった。

今は weekly-review.md inbox-review.md azure-devops-sync.md のように、「何を」「どうする」がファイル名でわかる ようにしている。

原則3: ルールは「禁止」より「推奨」で書く

～するな ～禁止 ばかりのルールファイルを書くと、AIの動きがガチガチに硬くなる。セキュリティ関連は禁止でいいけど、それ以外は「こうすること」という**推奨形式**のほうが柔軟に対応してくれる。

```
# ❌ 禁止ばかり
- pages/ ルーターを使うな
- Supabase を使うな
- rm コマンドを使うな

# ✅ 推奨 + 必要な禁止だけ
- App Router を使用すること
- データベースは MongoDB Atlas を使用すること
- ファイル削除は trash コマンドを使用すること (rm 禁止)
```

原則4: 1コマンド = 1タスクの原則

「デイリーノート作成 + Inbox整理 + タスク同期」を1つのコマンドにまとめたくなるけど、これは分けたほうがいい。

理由は3つ。途中で失敗したときに再実行しやすい。組み合わせを変えられる。そして **コマンドの名前が明確になる**。

複合ワークフローが必要ななら、skills/ を使う。

原則5: 定期的に棚卸しする

コマンドやルールは、使っていくうちに陳腐化する。「このルール、もう要らないな」「このコマンド、3ヶ月使っていないな」というものが出てくる。

月に1回くらい、`.claude/` ディレクトリを眺めて整理する時間を作るといい。僕は vault-master スキルの中に、この棚卸しも組み込んでいる。

明日から始める3ステップ

「35個もコマンド作るのが大変そう...」と思った人。大丈夫。最初は3つのステップだけでいい。

Step 1: CLAUDE.md を書く（5分）

Vault のルートに `CLAUDE.md` を作って、以下だけ書く。

```
# My Vault

## このVaultについて
（一言で説明）

## ディレクトリ構成
（主要フォルダだけ）

## 命名規則
（最低限のルール）
```

これだけで、Claude Code がVaultの構造を理解してくれるようになる。

Step 2: よく使う操作を1つだけコマンド化する（10分）

毎日やっている操作を1つ選んで、`.claude/commands/` にファイルを作る。

デイリーノートの作成でもいいし、Inboxの確認でもいい。 **一番面倒だと感じている操作** を選ぶのがコツ。

Step 3: 1週間使って rules/ を追加する（随時）

1週間使っていると、「Claude がこう判断してほしいのに」という場面が出てくる。そのたびに `rules/` にファイルを1つ追加する。

ルールは一気に作るんじゃなくて、 **実際に困ったときに1つずつ増やす** のが自然でいい。

おわりに

Obsidian が「考える脳」なら、Claude Code は「動く手足」みたいなもので。

`.claude/` ディレクトリを整えていくと、Vault が「ただのノートの集まり」から **「自分と一緒に動いてくれるシステム」** に変わっていく感覚がある。

35個のコマンドは一気にできたわけじゃなくて、「ここ面倒だな」「これ毎回やってるな」と感じるたびに1つずつ増えていった結果。だから、焦らず、自分のペースで育てていけばいいと思う。

この記事が、同じように「AIと一緒にナレッジ管理をしたい」と考えている人の何かのヒントになれば嬉しい。

はじめに — Obsidian に Claude Code を入れたら、35個のコマンドが生まれた

ノートは1000を超え、Daily Notes、タスクリスト、プロジェクトメモ、議事録...あらゆるものが Vault に溜まっていった。でも、不思議なことに、ノートが増えれば増えるほど「管理する時間」も増えていく。

毎朝のデイリーノート作成に5分。Inboxの整理に10分。週次レビューに30分。ノートを書く時間より、ノートを管理する時間のほうが長くなっている...そんな本末転倒な状態になっていた。

ある日、ふと思った。「Claude Code の `.claude/` ディレクトリって、コードベースじゃなくても使えるんじゃないか」と。

試しに Obsidian Vault のルートに `.claude/` を作って、いくつかコマンドを書いてみた。 `/daily` と打つだけでデイリーノートが生成される。 `/inbox-review` で未整理ノートを分析してくれる。

気づいたら、**コマンドが35個、ルールが18個、スキルが10個** に育っていた。

この記事では、Obsidian Vault に `.claude/` ディレクトリを設計するパターンと、実際に運用しているコマンド・ルール・スキルの実例を全部共有する。「コードベースじゃないプロジェクト」に Claude Code を組み込むという発想が、もしかしたら誰かの参考になるかもしれないので。

注意: この記事は2026年2月時点の Claude Code の仕様に基づいています。コード例は筆者の環境をサニタイズしたもので、そのままコピペしても動かない部分があります。自分の環境に合わせて調整してください。

`.claude/` ディレクトリの全体像

まず、全体の構造を見てみよう。

```
my-obsidian-vault/
├── CLAUDE.md                # プロジェクトの「自己紹介書」
├── .claude/
│   ├── commands/           # スラッシュコマンド（35個）
│   │   ├── daily.md
│   │   ├── weekly-review.md
│   │   ├── inbox-review.md
│   │   ├── mtg.md
│   │   ├── research.md
│   │   └── ...
│   ├── rules/              # AIの判断基準（18個）
│   │   ├── tech-stack.md
│   │   ├── naming.md
│   │   ├── timezone.md
│   │   └── ...
│   ├── skills/             # 複合ワークフロー（10個）
│   │   ├── vault-master/
│   │   │   └── SKILL.md
│   │   └── ...
└── settings.local.json     # ローカル設定
```

レストランに例えるとこうなる。

- **CLAUDE.md** = メニュー表。「うちの店はこういう店です」という自己紹介
- **commands/** = 注文ボタン。「これください」でパッと出てくる定型オペレーション
- **rules/** = 調理マニュアル。「うちではこう作る」という判断基準
- **skills/** = コース料理のレシピ。複数の工程を組み合わせた複雑なワークフロー

この4つの柱を使い分けることで、AIエージェントが「何をすべきか」「どう判断すべきか」「どんな手順で進めるべきか」を理解できるようになる。

CLAUDE.md — プロジェクトの「自己紹介書」を書く

CLAUDE.md は、Claude Code がセッション開始時に **最初に読むファイル**。ここにプロジェクトの全体像を書く。

何を書くべきか